

V3.0 · INTERNAL DOCUMENT

Multi-Tenant SaaS Launch Plan

A four-phase roadmap to take the current single-tenant SolidTech prototype into a centralized, self-serve SaaS platform supporting 20–100 paying tenants with Stripe billing, subdomain isolation, and a master administration suite.

PREPARED BY

Claude (Opus 4.6 · 1M)

FOR

Jason Hogan

GENERATED

April 15, 2026

SCOPE

Phases 1–4

v3.0 — Multi-Tenant SaaS (Direct Signups, Stripe Billing, Absorbed AI Costs)

Context

Jason's boss saw the current v2.3 tool (single-tenant prototype for SolidTech) and wants to turn it into a centralized SaaS platform where any company can sign up and get their own branded copy — instead of forking the repo and running a separate install per customer. One codebase, one deployment, tenant isolation via the existing `client_id` column.

The app was already built multi-tenant-ready at the code level: every table has `client_id`, every query filters by `$GLOBALS['client_id']`, branding is already per-client, RBAC is already in place. **The hardest architectural work is already done.** What's missing is the signup/billing/master-admin layer on top, plus a full data-isolation audit before anyone hands us a credit card.

This plan is scoped for **v3.0 — beta through commercial launch targeting 20–100 paying tenants**. Stays on SiteGround through beta; DigitalOcean App Platform migration is scheduled in as a separate phase to trigger when performance demands it, not upfront.

Locked decisions (from Jason 2026-04-15)

DECISION	CHOICE	IMPLICATION
Tenant model	Direct SaaS — one signup = one company	No agency/reseller layer. Matches current <code>clients</code> table 1:1. Simplest.
AI cost model	Absorb costs, bake into plans	Must meter AI usage per tenant + enforce per-plan hard limits or Jason's AI bill can be run up by a single abusive tenant.
Custom domains	No — subdomains only (<code>acme.yourtool.com</code>)	Wildcard DNS + wildcard SSL. Massively simpler than per-domain Let's Encrypt provisioning.
v3.0 scale target	20–100 paying tenants	Stay on SiteGround through beta, migrate to DO App Platform when strain shows. Need solid self-serve onboarding.

Architecture: Shared DB + `client_id` filtering

One MySQL database. One PHP codebase. Tenants isolated by column. What you already have, extended with a signup/billing/admin layer on top.

The existential risk is a missed `WHERE client_id = ?` = cross-tenant data leak. The first phase of this plan is a full isolation audit before writing any signup code, because fixing leaks is trivial while there's one tenant and nightmarish once there are fifty.

Branding already works per-tenant (`branding_settings` table), so the "each customer gets their own little world" feeling comes free — they see their own logo, colors, company name, custom tour, no mention of your master brand once signed up.

Phase 1 — Data Isolation Audit + Hardening

Goal: make the codebase safe to host multiple paying tenants. No new features. Pure defensive work.

1.1 Full SQL audit

Grep every `SELECT`, `UPDATE`, `DELETE` in `app/models/` + `app/controllers/` and classify each query:

-  Already filters by `client_id`
-  Missing `client_id` filter
-  Filters by `user_id` only (OK if `users.client_id` is then used, but risky)

Produce a table of every query and its tenancy status. Any  gets fixed in this phase. Expected count: 10–30 queries need fixes; most of the codebase is already correct per the existing `$GLOBALS['client_id']` pattern.

1.2 Generalize the `findForClient` pattern

`GeneratedReport::findForClient($id, $clientId)` is the correct pattern from Phase 3. Push this to the base `Model` class:

```
// core/Model.php
public function findForTenant(int $id, int $clientId): ?array
{
    return Database::fetch(
        "SELECT * FROM {$this->table} WHERE id = :id AND client_id = :cid LIMIT 1",
        ['id' => $id, 'cid' => $clientId]
    ) ?: null;
}

public function updateForTenant(int $id, int $clientId, array $data): void { /* ... */ }
public function deleteForTenant(int $id, int $clientId): void { /* ... */ }
```

Then audit every call to `Model::find($id)` / `Model::update($id, ...)` / `Model::delete($id)` in controllers and replace with the tenant-scoped variants. Ban raw-ID lookups in code review.

1.3 Cross-tenant isolation tests

New test file `tests/CrossTenantIsolationTest.php` (custom test runner, no PHPUnit — keep zero deps). Tests:

- Seed two tenants (A, B) with posts, reports, users, branding

- Log in as A → attempt to fetch B's post by raw ID → must return 404
- Attempt to edit B's post via `/posts/edit/{id}` POST → must return 404
- Attempt to share B's report → must return 404
- Attempt to view B's activity log → must return 404
- Attempt to upload a file to B's branding → must return 404
- Public shared report from B with valid token → A can view (correct — shared means shared)

Run before every v3.x deploy.

1.4 File upload isolation

Audit every upload path:

- `uploads/branding/`
- `uploads/reports/`
- `uploads/posts/` (or wherever post images land)

Current state: likely shared-flat. Move to `uploads/{client_id}/branding/`, `uploads/{client_id}/posts/`, etc. Write a one-shot migration script that walks existing files and fixes paths.

Every `move_uploaded_file` call must include `$clientId` in the destination path. Validate in a code-review pass.

1.5 HTTP-level tenant resolution

Right now the app sets `$GLOBALS['client_id'] = 1` because SolidTech is the only tenant. This changes in Phase 2, but the hook must already be in place:

```
// public/index.php bootstrap
$GLOBALS['client_id'] = TenantResolver::fromRequest();
```

New class `core/TenantResolver.php` :

- Inspects `$_SERVER['HTTP_HOST']` → extracts subdomain → looks up `clients.subdomain`
- Fallback to `$_SESSION['client_id']` for logged-in users (set at login)
- Returns `null` if no match → middleware redirects to the signup marketing page

In Phase 1 it keeps returning `1` for any host (backward compatible). Phase 2 activates real subdomain resolution.

1.6 Critical files — Phase 1

ACTION	FILE
CREATE	<code>core/TenantResolver.php</code>
CREATE	<code>tests/CrossTenantIsolationTest.php</code>
CREATE	<code>database/migrations/006_file_isolation.sql</code> (no schema change — just documents the storage migration)
EDIT	<code>core/Model.php</code> — add <code>findForTenant</code> , <code>updateForTenant</code> , <code>deleteForTenant</code>
EDIT	<code>public/index.php</code> — swap hardcoded <code>client_id = 1</code> for <code>TenantResolver::fromRequest()</code> (still returns 1 in Phase 1)
EDIT	10–30 model/controller files identified by the audit — swap raw ID lookups for tenant-scoped variants
EDIT	Every upload handler — inject <code>\$clientId</code> into destination path

1.7 Phase 1 verification

1. Run the cross-tenant isolation test suite → all pass with 2 seeded tenants
2. Seed a second tenant locally, log in as A, manually probe B's resources via URL manipulation → 404s across the board
3. Confirm SolidTech prod still works normally (no regressions) after deploying the tenant-scoped base model methods
4. Confirm file uploads land in `uploads/1/...` on prod after the storage migration
5. Run the query audit spreadsheet through code review subagent before deploying

Phase 2 — Signup Flow + Subdomain Routing + Master Admin

Goal: any company can sign up and get their own branded copy at `{their-slug}.yourtool.com`. Master admin (you) can see + manage them all.

2.1 Database

```
-- Extend the existing clients table
ALTER TABLE clients
  ADD COLUMN subdomain VARCHAR(63) NOT NULL UNIQUE AFTER name,
  ADD COLUMN status ENUM('trial','active','past_due','suspended','canceled') NOT NULL DEFAULT 'trial',
  ADD COLUMN trial_ends_at TIMESTAMP NULL,
  ADD COLUMN stripe_customer_id VARCHAR(64) NULL,
  ADD COLUMN stripe_subscription_id VARCHAR(64) NULL,
  ADD COLUMN plan VARCHAR(20) NOT NULL DEFAULT 'starter',
  ADD COLUMN signup_email VARCHAR(255) NULL,
  ADD COLUMN signup_ip VARCHAR(45) NULL;

-- Master admin account — lives outside any tenant
CREATE TABLE master_admins (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  first_name VARCHAR(100) NOT NULL,
  last_login_at TIMESTAMP NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Usage metering — rolled up daily per tenant
CREATE TABLE usage_counters (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  client_id INT UNSIGNED NOT NULL,
  period_start DATE NOT NULL,          -- first day of billing cycle
  ai_generations INT UNSIGNED DEFAULT 0,
  image_jobs INT UNSIGNED DEFAULT 0,
  posts_published INT UNSIGNED DEFAULT 0,
  emails_sent INT UNSIGNED DEFAULT 0,
  UNIQUE KEY uk_client_period (client_id, period_start),
  INDEX idx_client (client_id)
);
```

2.2 Signup flow (public, unauthenticated)

Route: `/signup` on the marketing subdomain (or bare `yourtool.com/signup`)

Form fields:

- First name, last name
- Company name
- Desired subdomain (validated: `/^[a-z0-9][a-z0-9-]{2,30}[a-z0-9]$/` , blacklist: `www` , `api` , `admin` , `app` , `mail` , `marketing` , etc.)
- Email
- Password (min 8 chars — same rules as current password reset)

Behavior:

1. Validate inputs → check subdomain is free in `clients.subdomain`
2. Create `clients` row with `status='trial'` , `trial_ends_at = NOW() + 14 days` , `plan='starter'`
3. Create first `users` row with `role='admin'` , scoped to new `client_id`
4. Copy default `branding_settings` row (neutral placeholder logo + primary color)
5. Copy default `content_themes` (the "starter pack" we'll seed)
6. Copy default `art_direction_settings`
7. Send welcome email: "Your account is ready at `{subdomain}.yourtool.com` — log in with the password you just set"
8. Log activity to a new `master_activity_log` table (not the per-tenant one)
9. Redirect to `https://{subdomain}.yourtool.com/login`

No Stripe involvement at signup — trial doesn't require a card. Stripe kicks in at the trial-ending upgrade flow (Phase 3).

Safety rules:

- Rate limit by IP (5 signups / hour / IP) — prevents subdomain squatting
- No card data collected in this flow (never touches PCI scope)
- Master admin gets notified via email on every new signup during beta

2.3 Subdomain routing

DNS: wildcard `.yourtool.com` pointing to SiteGround IP. Already supported by SiteGround shared hosting (just needs a `A` record in cPanel).

SSL: wildcard cert for `*.yourtool.com` . SiteGround supports wildcard Let's Encrypt on shared hosting — free, auto-renewing. Activate once.

Application: `core/TenantResolver::fromRequest()` activates in Phase 2:

```
public static function fromRequest(): ?int
{
    $host = $_SERVER['HTTP_HOST'] ?? '';
    // Strip port if present
    $host = preg_replace('/:~d+$/ ', '', $host);

    // Marketing root – no tenant
    if ($host === 'yourtool.com' || $host === 'www.yourtool.com') {
        return null;
    }

    // Parse subdomain
    if (preg_match('/^([a-z0-9-]+)\.yourtool\.com$/ ', $host, $m)) {
        $subdomain = $m[1];
        // Reserved subdomains route to master admin
        if (in_array($subdomain, ['admin', 'api', 'app'], true)) {
            return null;
        }
        $row = Database::fetch(
            "SELECT id, status FROM clients WHERE subdomain = :sub LIMIT 1",
            ['sub' => $subdomain]
        );
        if (!$row) return null;
        if ($row['status'] === 'suspended' || $row['status'] === 'canceled') {
            // Middleware shows a "your account is suspended" page
            $GLOBALS['tenant_suspended'] = true;
        }
        return (int) $row['id'];
    }
    return null;
}
```

2.4 Master admin dashboard (expanded)

Route: `admin.yourtool.com` (or `yourtool.com/master`)

Auth: separate login form that checks `master_admins` table, **not** `users`. Session key `$_SESSION['master_admin_id']`. Enforce **2FA (TOTP)** from day one — this is the god-mode account, no exceptions.

2.4.1 Overview dashboard (landing page)

The first thing the master admin sees — at-a-glance business health:

- **MRR / ARR counters** — live numbers from Stripe, animated count-up (reuse the pattern from the reports page)
- **Active tenants / trialing / past-due / suspended** splits as stat cards
- **New signups last 7 days** with a mini sparkline
- **Churn last 30 days** — absolute count + percentage
- **Top 5 highest-usage tenants this period** — flags potential abuse early
- **System health indicators** — cron last-ran-at, background job queue depth, error rate last 24h, average response time
- **Activity stream** — the last 50 master-level events across all tenants (signup, upgrade, cancel, payment failed, tenant suspended, master login, impersonation)

2.4.2 Tenants list

Table of all `clients` rows, sortable/filterable:

COLUMN	NOTES
Subdomain	Click → tenant detail
Company name	From branding_settings
Plan	Color-coded badge
Status	trial / active / past_due / suspended / canceled
MRR	Plan price, \$0 if trialing
Seats used / total	From <code>users</code> count
Usage this period	AI generations + posts + image jobs as mini bars against plan limit
Trial ends	Only for trialing tenants, days remaining
Last activity	"2 hours ago" from activity_logs — instantly spot dead accounts
Signup date	For cohort filtering
Actions	... menu: view / impersonate / suspend / edit plan / refund / note

Filters: status, plan, signup date range, usage-percent threshold, "over plan limit", "trial ending this week", "no activity 14+ days", free-text search on subdomain + email + company name.

Exportable to CSV.

2.4.3 Tenant detail page

Everything you might need to answer a support ticket in one view, no digging:

Top section — company profile:

- Logo, company name, subdomain, primary brand color preview
- Signup date, signup IP, signup user agent
- Current plan, Stripe customer ID (links to Stripe dashboard), next invoice amount + date
- Trial countdown if applicable

Users tab — all `users` rows scoped to this client_id with name, email, role, last login, status. Actions: send password reset email, deactivate user.

Usage tab — per-period rollups from `usage_counters` :

- Bar chart of AI generations / posts / images / emails over the last 6 months
- Current period usage vs limits with progress bars
- Cost to you estimate (based on your OpenRouter + Kie.ai + Zernio rates × their usage)
- **Margin alert** if their AI cost is approaching their plan price

Billing tab:

- Stripe invoice list (last 12 months) with amount, status, date, download link
- Payment history
- Refund button (requires reason + master admin password re-entry)
- "Issue credit" for one-off concessions
- Manual plan change (with Stripe subscription update)

Activity tab — the tenant's own activity log, read-only

Content tab — quick stats: post count, report count, theme count, last generated post preview

Support notes tab — free-form master-admin-only notes on this tenant (e.g., "Reached out about Enterprise pricing on 2026-05-01"). Timestamped, signed by the master admin who wrote it.

Danger zone (always at the bottom, scrolled to):

- Suspend tenant (requires typing their subdomain to confirm)
- Unsuspend tenant
- Extend trial by N days
- Force cancel subscription

- Soft-delete tenant (30-day grace)
- Hard-delete tenant + all data (requires typing full company name + master password + a 10-second confirmation countdown)

2.4.4 Impersonation — do this right

"Log in as tenant admin" button in the tenant detail page. This is the feature you'll use every day for support. Requirements:

- Separate session variable: `$_SESSION['impersonating_as_user_id']` + `$_SESSION['impersonator_master_id']`
- Tenant view renders normally but a **bright red banner across the top** shows "You are signed in as Jane Doe from Acme Corp. [Return to Master Admin]" — no chance of forgetting
- **Read-only mode toggle** — master admin can choose to impersonate in read-only mode (can see everything but all mutating forms are disabled)
- **All impersonation actions are logged twice:** once to `master_activity_log` ("Master admin jhogan impersonated Acme Corp, action: post_deleted") and once to the tenant's own `activity_logs` ("Master admin impersonated this account and deleted post #42")
- Impersonation session expires after 60 minutes — can't "forget" to log out
- Tenant admin gets an email notification within 5 minutes: "A member of our support team accessed your account at 14:32 to help with a recent ticket. If this wasn't expected, contact us."
- A tenant can opt out of impersonation in their privacy settings — master admin sees "impersonation blocked by tenant" and must contact them first

This is the #1 thing competitors get wrong. Transparency = trust.

2.4.5 Financial dashboard

A dedicated page just for money, because your boss is going to want this:

- **MRR / ARR:** current value + 12-month trend line
- **New MRR / Expansion MRR / Churned MRR / Contraction MRR** — the four SaaS metrics that matter
- **Churn rate** — monthly + annualized, customer churn vs revenue churn
- **LTV:CAC ratio** (once you know your CAC)
- **Cohort retention heat-map** — N months of cohorts × N months of retention
- **Plan distribution** — pie chart of tenants per plan
- **Top-expanding / top-shrinking tenants** this period
- **Failed payment recovery rate**
- **Revenue by month** as a bar chart

- All of these pull from `stripe_invoices` (new table we cache from webhooks) so Stripe Dashboard isn't the source of truth for the master dashboard

2.4.6 Abuse / fraud detection page

Things to catch before they hurt you:

- **New-signup watchlist** — last 50 signups with IP, country, email domain (gmail/disposable), subdomain, "signup velocity" per IP
- **Disposable email flag** — free list of disposable domains (mailinator, 10minutemail, etc.) — flag signups using them
- **Duplicate IP signups** — same IP created N accounts in 24 hours
- **Usage anomaly detector** — tenants whose AI usage in the last 24h is > 5x their historical daily average (early warning for prompt abuse / API key sharing / runaway scripts)
- **Rate-limit violations** — tenants hitting endpoint rate limits, which usually means misconfigured integrations or bots
- **Action buttons:** one-click suspend, one-click ban by IP/email, one-click lock-to-read-only

2.4.7 Support / communication hub

- **Recent support activity** — latest master-admin notes across all tenants
- **Announce feature** — push a banner message to all tenant dashboards ("Scheduled maintenance Saturday 2am EST"). Scoped options: all tenants, specific plan tier, specific tenant.
- **Email all tenants on a plan** — send a one-off broadcast (e.g., "New Pro tier feature: ..."). Uses EmailService with the master admin's sending identity, not any tenant's SMTP.
- **Incident status page toggle** — flip a flag that puts `status.yourtool.com` (or a top banner) into "investigating" / "resolved" mode — reassures users during outages

2.4.8 System config / feature flags

- **Feature flags** — toggle features on/off without deploying (e.g., `ENABLE_BILLING`, `ENABLE_SIGNUPS`, `ENABLE_AI_IMAGE_GEN`). Stored in a new `system_flags` table, checked at runtime.
- **Plan catalog editor** — edit plan limits + prices without a code deploy (still have to update Stripe prices manually though)
- **Global rate limits** — per-tenant and per-endpoint, tunable live
- **Maintenance mode toggle** — turns the whole app into a "we'll be right back" page except for the master admin
- **Kill switch for any tenant** — nuclear option: instant global lockout of a specific tenant (e.g., legal request, active abuse)

2.4.9 Audit log for master admin actions

Every master-admin action writes to `master_activity_log` . Searchable by:

- Master admin who took the action
- Action type (impersonate, suspend, refund, plan_change, delete_tenant, etc.)
- Tenant affected
- Date range
- IP address of master admin at the time

This becomes critical the moment you hire a second master admin or get a legal discovery request.

2.4.10 Master admin UX principles

- **Read-mostly by default** — destructive actions always behind a "type the name to confirm" gate
- **No silent actions** — every impersonation, refund, plan change, suspension shows a toast + writes to audit log
- **Two-person rule on hard deletes** — hard-delete a tenant requires a second master admin to approve within 24h (once you have more than one master)
- **Separate physical domain** — `admin.yourtool.com` is a different subdomain from any tenant subdomain so it can have its own IP allowlist / VPN / extra auth layer
- **Never expose tenant data in master URLs** — use opaque UUIDs in admin URLs, not sequential IDs, to prevent shoulder-surfing attacks at conferences

2.4.11 New database tables required for a real master admin

```
-- Master-level audit log (separate from per-tenant activity_logs)
CREATE TABLE master_activity_log (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  master_admin_id INT UNSIGNED NOT NULL,
  master_admin_email VARCHAR(255) NOT NULL,          -- snapshot
  action VARCHAR(60) NOT NULL,                      -- 'tenant_suspended', 'impersonation_start', 'plan_c
  target_client_id INT UNSIGNED NULL,                -- NULL for system-wide actions
  target_user_id INT UNSIGNED NULL,
  description VARCHAR(500) NULL,
  metadata JSON NULL,
  ip_address VARCHAR(45) NULL,
  user_agent VARCHAR(500) NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_master (master_admin_id, created_at),
  INDEX idx_client (target_client_id, created_at),
  INDEX idx_action (action)
);

-- Feature flags, tuned live without deploy
CREATE TABLE system_flags (
  flag_key VARCHAR(60) PRIMARY KEY,
  flag_value VARCHAR(500) NOT NULL,
  description VARCHAR(500),
  updated_by INT UNSIGNED NULL,                      -- master_admin_id
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Master-level support notes per tenant
CREATE TABLE tenant_notes (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  client_id INT UNSIGNED NOT NULL,
  master_admin_id INT UNSIGNED NOT NULL,
  note TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_client (client_id, created_at)
);

-- Cached Stripe invoices so the master dashboard doesn't hit Stripe API on every page load
CREATE TABLE stripe_invoices (
  id VARCHAR(64) PRIMARY KEY,                        -- Stripe invoice ID
  client_id INT UNSIGNED NOT NULL,
  amount_cents INT NOT NULL,
  currency VARCHAR(3) NOT NULL,
  status VARCHAR(20) NOT NULL,                       -- paid, open, uncollectible, void
  paid_at TIMESTAMP NULL,
  created_at TIMESTAMP NOT NULL,
  hosted_invoice_url VARCHAR(500) NULL,
```

```
pdf_url VARCHAR(500) NULL,  
INDEX idx_client_created (client_id, created_at)  
);  
  
-- Abuse detection signals  
CREATE TABLE abuse_signals (  
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  client_id INT UNSIGNED NULL,  
  signal_type VARCHAR(40) NOT NULL,          -- 'usage_spike', 'disposable_email', 'ip_velocity',  
  severity ENUM('info','warning','critical') NOT NULL DEFAULT 'info',  
  details JSON NULL,  
  resolved_at TIMESTAMP NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  INDEX idx_client_created (client_id, created_at),  
  INDEX idx_severity_created (severity, created_at)  
);  
  
-- Impersonation opt-out flag on tenant side  
ALTER TABLE clients  
  ADD COLUMN allow_impersonation TINYINT(1) NOT NULL DEFAULT 1 AFTER plan;  
  
-- 2FA secret storage for master admins  
ALTER TABLE master_admins  
  ADD COLUMN totp_secret VARCHAR(64) NULL,  
  ADD COLUMN totp_backup_codes JSON NULL,  
  ADD COLUMN totp_enabled_at TIMESTAMP NULL;
```


2.5 Critical files — Phase 2

ACTION	FILE
CREATE	<code>database/migrations/007_multitenant_signup.sql</code>
CREATE	<code>app/controllers/SignupController.php</code>
CREATE	<code>app/controllers/MasterAdminController.php</code>
CREATE	<code>app/models/MasterAdmin.php</code>
CREATE	<code>app/models/UsageCounter.php</code>
CREATE	<code>app/services/TenantProvisioningService.php</code> — the code that creates a new tenant from a signup (copies defaults, seeds content)
CREATE	<code>app/services/UsageMeterService.php</code> — increment counters on AI calls, post publishes, etc.
CREATE	<code>app/views/signup/index.php</code> (marketing-page style)
CREATE	<code>app/views/master/tenants.php</code>
CREATE	<code>app/views/master/tenant_detail.php</code>
CREATE	<code>app/views/master/dashboard.php</code>
CREATE	<code>app/views/emails/welcome.php</code>
CREATE	<code>app/views/suspended.php</code> — shown to suspended tenants
EDIT	<code>core/TenantResolver.php</code> — activate real subdomain parsing
EDIT	<code>config/routes.php</code> — add signup + master admin routes
EDIT	<code>public/index.php</code> — route-before-client-id resolution for master admin subdomain
EDIT	<code>app/services/AIService.php</code> — call <code>UsageMeterService::incrementGeneration(\$clientId)</code> after every generation, enforce plan limits before calling OpenRouter
EDIT	<code>app/controllers/PostController.php</code> — meter post publishes

2.6 Phase 2 verification

1. Visit `yourtool.com/signup` → fill form → new subdomain `test1.yourtool.com` reachable, shows default branding, first admin can log in
 2. SolidTech's existing subdomain still routes correctly (`solidtech.yourtool.com`) — zero regression
 3. Master admin dashboard at `admin.yourtool.com` shows all tenants
 4. Impersonation: click into a tenant as master admin → land in their dashboard → confirm activity log shows "Master admin impersonated" event
 5. Suspend a test tenant → their subdomain returns the suspended page → unsuspend → normal access restored
 6. Hit the AI generation limit on a Starter trial → plan limit enforced before OpenRouter is called → user sees upgrade prompt
 7. Rate-limit test: attempt 6 signups from the same IP in an hour → 6th blocked
-

Phase 3 — Stripe Billing + Plan Gating

Goal: tenants can upgrade from trial to a paid plan via Stripe Checkout. Failed payments degrade to read-only. Self-serve everything.

3.1 Plan catalog

Hardcoded in `config/plans.php`:

```
return [  
    'starter' => [  
        'name' => 'Starter',  
        'price_monthly' => 29,  
        'stripe_price_id' => 'price...',  
        'limits' => [  
            'ai_generations_per_month' => 100,  
            'posts_per_month' => 60,  
            'image_jobs_per_month' => 30,  
            'seats' => 2,  
        ],  
    ],  
    'pro' => [  
        'name' => 'Pro',  
        'price_monthly' => 99,  
        'stripe_price_id' => 'price...',  
        'limits' => [  
            'ai_generations_per_month' => 500,  
            'posts_per_month' => 300,  
            'image_jobs_per_month' => 150,  
            'seats' => 10,  
        ],  
    ],  
    'agency' => [  
        'name' => 'Agency',  
        'price_monthly' => 299,  
        'stripe_price_id' => 'price...',  
        'limits' => [  
            'ai_generations_per_month' => 2000,  
            'posts_per_month' => 1500,  
            'image_jobs_per_month' => 750,  
            'seats' => 30,  
        ],  
    ],  
];
```

These tiers are the 2026 starting point Jason and I discussed. Adjust based on beta feedback.

3.2 Stripe integration

Stripe PHP SDK: drop-in to `vendor/stripe/` like mPDF was in Phase 2 (~1 MB, no Composer). Current version `stripe-php 14.x`.

Flow:

- 1. Tenant in trial clicks "Upgrade" in their dashboard
- 2. Backend creates a Stripe Checkout Session with `client_id` in metadata + success/cancel URLs
- 3. Redirect to Stripe-hosted checkout
- 4. On success, Stripe webhook `checkout.session.completed` → find tenant by `client_id` metadata → flip to `status='active'`, store `stripe_customer_id` + `stripe_subscription_id` + `plan`
- 5. User lands back on dashboard with a "Welcome to Pro!" toast

Customer Portal: `/billing` route in the app → creates a Stripe Billing Portal session for the tenant's `stripe_customer_id` → redirects. Stripe handles plan changes, invoice downloads, payment method updates, cancellation. **Free feature from Stripe.** Ship it.

Webhooks (`/stripe/webhook` route, CSRF-exempt, signature-verified):

EVENT	ACTION
<code>checkout.session.completed</code>	Activate tenant, store subscription ID + plan
<code>customer.subscription.updated</code>	Sync plan change, update limits
<code>invoice.payment_failed</code>	Set <code>status='past_due'</code> , send email to tenant admin
<code>invoice.payment_succeeded</code>	Clear <code>past_due</code> if set
<code>customer.subscription.deleted</code>	Set <code>status='canceled'</code> , retain data for 30 days, then cron-delete

3.3 Plan gating middleware

Every AI generation, image job, post publish, email send checks `UsageMeterService::canPerform($clientId, 'ai_generations')` which reads the current period's `usage_counters` row and the tenant's plan limit. Over limit = friendly error with "Upgrade" CTA.

Starter trial gets the Starter limits (100 generations) so a trialling tenant can't run up the bill.

Critical safety: hard-limit at 120% of plan to prevent race conditions from blowing out the budget. Soft-warn at 80%, hard-block at 120% (not 100%, because real-world counters lag slightly). Alert master admin if a tenant goes past 100%.

3.4 Dunning + suspension

Day 0: payment fails → `status='past_due'`, email tenant admin "Update your payment method" **Day 3:** second reminder email **Day 7:** feature-block — tenant can still log in and read, but can't generate, post, or publish **Day 14:** `status='suspended'` — subdomain shows suspended page until resolved **Day 44** (30 days after suspension): cron deletes all tenant data (GDPR-compliant; tenant gets 30-day data export warning)

All of this lives in a new `cron/dunning.php` that runs daily.

3.5 Critical files — Phase 3

ACTION	FILE
CREATE	<code>vendor/stripe/</code> (dropped in, no Composer)
CREATE	<code>config/plans.php</code>
CREATE	<code>app/services/BillingService.php</code> — Stripe SDK wrapper (create_checkout, create_portal, handle_webhook)
CREATE	<code>app/controllers/BillingController.php</code> — <code>/billing/upgrade</code> , <code>/billing/portal</code> , <code>/stripe/webhook</code>
CREATE	<code>app/views/billing/upgrade.php</code> — 3-card plan comparison
CREATE	<code>app/views/emails/payment_failed.php</code>
CREATE	<code>app/views/emails/trial_ending.php</code>
CREATE	<code>app/views/emails/subscription_canceled.php</code>
CREATE	<code>cron/dunning.php</code>
CREATE	<code>database/migrations/008_stripe_columns.sql</code> (may be a no-op if Phase 2 added them already)
EDIT	<code>app/services/AIService.php</code> — check <code>UsageMeterService::canPerform</code> before calling OpenRouter
EDIT	<code>app/services/ImageJobService.php</code> — same for image generation
EDIT	<code>app/controllers/PostController.php</code> — check post limit before scheduling/publishing
EDIT	<code>app/services/EmailService.php</code> — check email limit before sending
EDIT	<code>app/views/layouts/main.php</code> — plan badge + usage meter in topbar

3.6 Phase 3 verification

1. Trial tenant clicks Upgrade → Stripe Checkout loads → pay with test card `4242 4242 4242 4242` → webhook fires → tenant flips to `active` with Pro plan → toast shows

2. `/billing/portal` → Stripe Customer Portal loads → change plan to Starter → webhook updates plan → limits adjust
 3. Payment fails (test card `4000 0000 0000 0341`) → webhook flips to `past_due` → email sent → day 7 simulation blocks AI → day 14 simulation suspends
 4. Starter tenant hits 100 generations → 101st is blocked with upgrade CTA → verify OpenRouter is never called after the block
 5. Starter tenant tries to race past the limit with 20 concurrent requests → none break the 120% hard ceiling
 6. Cancel subscription via Portal → webhook sets `canceled` + schedules 30-day data deletion → cron runs → data gone
 7. Full end-to-end: fresh signup → 14-day trial → upgrade to Pro → downgrade to Starter → cancel → resume (checkout new subscription) → suspend for non-payment → recover
-

Phase 4 — Hosting Migration to DigitalOcean (TRIGGERED, NOT SCHEDULED)

Goal: have a migration plan ready to execute when SiteGround strain appears. Not a scheduled phase — only triggered by measurable symptoms.

4.1 Trigger conditions

Execute this phase when **any two** of the following are true in prod for 3+ consecutive days:

- Average response time on `/dashboard` > 1.5s
- SiteGround cron running beyond its throttle window (reported as `cron.log` errors)
- CPU quota warnings from SiteGround cPanel
- More than 30 active paying tenants
- `image_jobs` backlog > 100 at any point
- A single tenant's AI usage causes user-visible latency for others

4.2 Target: DigitalOcean App Platform

- **PHP app** deployed from GitHub via App Platform (auto-deploy on push to `main`)
- **Managed MySQL** (\$15/mo starter, scales up to HA)
- **Managed Redis** (\$15/mo) — for rate limiting + session storage + job queue
- **Spaces** (S3-compatible) — for uploads, move off SiteGround filesystem. ~\$5/mo for 250GB.
- **Background job worker** — App Platform supports worker services. `ImageJobWorker.php` runs continuously instead of via cron.
- **Total:** ~\$50/mo to start, scales to ~\$150 at 100 tenants. Pays for itself at 2 paying tenants.

4.3 Migration steps

1. Dump SiteGround MySQL → import to managed MySQL
2. Move `public/uploads/` → DO Spaces, update all upload paths to use Spaces URLs (signed URLs for private files)
3. Add `core/Storage.php` abstraction so upload/fetch code doesn't care if it's local or S3
4. Deploy to App Platform staging env → smoke test all phases → cut DNS over (wildcard `*.yourtool.com` → App Platform LB)
5. Keep SiteGround running in parallel for 72h as hot rollback

6. Decom SiteGround once DO is proven

4.4 What does NOT change

- PHP codebase is unchanged (App Platform runs standard PHP 8.x)
- MySQL schema is unchanged
- Stripe integration is unchanged
- All tenants are unaffected by the move (they see the same subdomains)

Migration is ~1 weekend of work once triggered.

Cross-phase concerns

Data isolation is non-negotiable

Phase 1 blocks everything. No Phase 2 signup form ships until the cross-tenant tests all pass.

Stripe safety rules

Per my hard safety constraints AND PCI compliance:

- **Never** store, type, or handle card numbers in the app. Stripe Checkout is hosted by Stripe.
- **Never** auto-charge a customer. All charges go through Stripe's authenticated flow.
- Webhooks MUST verify the Stripe signature (`Stripe-Signature` header) before trusting any event payload — otherwise anyone can POST fake events to `/stripe/webhook` and flip tenants to `active` without paying.
- Store `stripe_customer_id` + `stripe_subscription_id` only. Never card fingerprints, last-4s, or anything card-related.

Master admin impersonation audit

Every impersonation event writes to `master_activity_log` + the impersonated tenant's own `activity_logs`. Tenant admins can see "Master admin logged in as you at 14:32" — no secret backdoor.

Email deliverability at scale

SiteGround SMTP is fine for 10 tenants sending a few emails each. For 100 tenants, batch reports, dunning notices, invitation emails → move outbound mail to a proper transactional service (Postmark, SendGrid, Amazon SES). EmailService already supports multiple providers — just switch the default in `config/env.php` on migration day.

Tenant onboarding experience

The current Wizard (scan → themes → branding) already exists. For v3.0:

- Run it automatically on first login after signup (force it, don't let them skip)
- Add a progress bar showing "3 of 5 steps to a ready account"
- After the wizard: 30-second video tour of the app (embedded YouTube, unlisted)
- Pre-seed 3 example posts + 1 example report so the new tenant's dashboard isn't empty and depressing on day 1
- "Book a 15-minute onboarding call" CTA for Pro/Agency tier signups — use Cal.com embed

Tenant self-service account management

Separate from Stripe Customer Portal, these are features that live inside the tenant app itself:

- **Team members page** — invite/remove/change role for their own users (you already have most of this as `app/views/users/`)
- **API keys** (v3.1+) — for tenants to integrate the app with their own tools
- **Webhook subscriptions** (v3.1+) — for tenants to get notified of post publish / failure events
- **Account settings** — change company name, subdomain (one free change per 90 days), timezone, locale
- **Data export** — download-everything button in settings → triggers a background job → emails a zip when ready
- **Delete my account** — requires typing the company name, warns about 30-day grace period, cancels Stripe subscription, starts the deletion cron

Status page + uptime monitoring

- Subscribe to an external status page service (StatusPage, Instatus, or self-hosted Cachet) at `status.yourtool.com`
- Automated uptime checks from external pingers (UptimeRobot free tier, or Betterstack) hitting `/healthz`, `/api/stats`, `/shared/test-token` (Phase 3 adds a known-good token for monitoring)
- Post incidents manually during outages; auto-post based on uptime checks
- Link to status page from the top of every tenant dashboard during an active incident

Observability you'll need at 20+ tenants

- **Error tracking:** Sentry free tier (5K events/month). Wire up to both PHP exceptions and browser JS errors. Group by tenant.
- **Performance:** New Relic free tier or SigNoz self-hosted. Track p50/p95 response time per route per tenant.

- **Log aggregation:** ship `cron.log` + `error.log` to a central location so they survive SiteGround log rotation. BetterStack Logs free tier works.
- **Query performance:** enable MySQL slow query log with threshold 500ms. Review weekly.
- **Business metrics dashboard:** rolled-up SQL queries into a nightly-refreshed `metrics_daily` table for the master admin financial dashboard (saves hammering Stripe/MySQL live on every page load)

Security hardening you'll need to ship alongside v3.0

- **HTTP security headers:** CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy. Set via `.htaccess` on SiteGround, via App Platform config on DO.
- **Rate limiting:** one bucket per endpoint per IP (already partially in place for view counters, needs to be formalized as middleware)
- **SQL injection audit:** one more pass through all queries for any string concatenation (should be zero — the PDO pattern you use is safe, but a final audit is cheap)
- **XSS audit:** all user-controlled strings in templates must go through `htmlspecialchars()`. The reports work already did this for branding colors; do a sweep on anything newly added.
- **CSRF:** every state-changing POST already uses `$_SESSION['csrf_token']`. Verify no new routes forget.
- **Password policy:** bump min length to 10, require one uppercase + one digit, check against haveibeenpwned.com password API before accepting new passwords. Free API, no key needed.
- **Session hijacking:** rotate session ID on every login + role change + impersonation start/end. Set `session.cookie_httponly = 1` + `session.cookie_secure = 1` + `session.cookie_samesite = Lax`.
- **Bot protection on signup:** hCaptcha (free) on the public signup form. Blocks 95% of automated signups.
- **Secrets management:** move all API keys out of `config/env.php` into SiteGround environment variables (or DO App Platform env config). Never commit secrets.

Legal / compliance checklist

- **Terms of Service** — template from Common Paper or Clerky, have a lawyer review before taking money. Needs clauses on AI-generated content ownership, data retention, limitation of liability, payment disputes.
- **Privacy Policy** — GDPR + CCPA compliant. Cover what data you collect, why, how long, who you share with (OpenRouter, Kie.ai, Stripe, your email provider).
- **Data Processing Agreement (DPA)** — required for EU customers. Template from Iubenda or similar.
- **Cookie banner** — you technically need one for EU users. Keep it minimal (essential cookies only is the easy mode — just session + CSRF cookies, no tracking).

- **SOC 2 / ISO 27001** — don't worry about these until an enterprise prospect asks. That's a 6-month + \$20K undertaking.
- **AI transparency** — disclose that posts are AI-generated in your ToS and in the UI (the existing "AI Generated" badge on reports is a good start). Some platforms (Meta, LinkedIn) now require this.

GDPR / data retention

- Tenant cancels → 30-day grace period → hard delete all rows with that `client_id` via a purge cron
- Tenant requests data export → master admin tool generates a zip of all their posts, branding, reports, activity log, uploads
- Privacy policy + terms of service pages required before taking real money. Draft from a template + have Jason's boss lawyer-review before launch.

Backward compatibility with SolidTech

SolidTech is the existing single-tenant customer and must keep working through every phase:

- Phase 1: zero user-visible changes, only code hardening
- Phase 2: SolidTech gets assigned `subdomain='solidtech'`, all URLs start resolving via `solidtech.yourtool.com`. Old direct domain (`marketing.aiagentprojects.com/social`) keeps working via a catch-all redirect.
- Phase 3: SolidTech is put on a free "internal" plan that bypasses billing + limits. Master admin can toggle this.
- Phase 4: they come along on the migration like any other tenant.

Rollback

Each phase is a finite set of files and one migration. Rollback = restore files from git + run the migration's `-- ROLLBACK:` block + FTP push. Stripe is the scariest to roll back — keep a feature flag (`ENABLE_BILLING=false` in env) that disables all Stripe routes and falls back to trial-everyone mode if something breaks.

Execution order

Phases are **sequential and independently deployable**. Each is its own ~1-week chunk, each deploys to prod on its own, each has a code review subagent pass before going live.

Recommended pacing:

1. Phase 1 (isolation audit) — **do this next session**, ~1 week

2. Phase 2 (signup + subdomain + master admin) — ~2 weeks
3. Phase 3 (Stripe billing + plan gating) — ~1.5 weeks
4. Beta invites to 5 friendly customers, fix what breaks, then open cold signups
5. Phase 4 (DO migration) — triggered, not scheduled, probably 2–3 months post-launch

Total to beta: ~5–6 weeks of focused work on the current stable codebase.

Pre-execution blockers

Before any of these phases start, Jason needs to:

1. **Buy the production domain** (e.g., `solidsocial.io` or whatever the final brand is). Current `marketing.aiagentprojects.com/social` isn't viable for multi-tenant SaaS.
2. **Have his boss lawyer-review** the ToS + Privacy Policy template before Phase 3 goes live.
3. **Set up Stripe account** (Jason / his boss — not something I can do) in test mode so Phase 3 can be developed against real Stripe infra.
4. **Decide pricing for real** — the \$29/\$99/\$299 tiers in this plan are a starting point, not a commitment.
5. **Decide the master admin email + 2FA setup** — that's the account with god-mode over everyone, keep it locked down.